Research — Faugus Launcher

Nov 4, 2026 · @Tim

Summary

Faugus is the reference for running standalone Windows games: an MIT-licensed Python launcher that starts games through umu-launcher and Proton, with per-game prefixes, a Proton manager, SteamGridDB art and Steam shortcut export. Release 2.0 (July 2026) ported it from GTK3 to GTK4 via PyGObject, but it uses libadwaita only for `Adw.StyleManager`, so its windows and dialogs are plain GTK. It offers four library modes, including a console-style Carousel with a hero banner, and a `--console` fullscreen gamepad mode via libmanette. Development is very active (2.5.7 on 2026-11-03), and its plain-JSON library plus `--game <id>` CLI make it easy for another app to import and launch its games.

Project facts

Faugus is a single-maintainer MIT project shipping roughly every two weeks since its GTK4 port.

Field	Value
Name, app ID	Faugus, `io.github.Faugus.faugus-launcher`
Repo	Faugus/faugus-launcher
Latest release	2.5.7, 2026-11-03 (Flathub still on 2.5.6 when checked)
Cadence	2.0.0 on 2026-07-20, then 2.2 (Aug 27), 2.3 (Sep 10), 2.4 (Sep 24), 2.5.0 (Oct 15), 2.5.x to Nov 3
Licence	MIT
Stack	Python 3, PyGObject, GTK 4, `Adw` 1 (StyleManager only), `Manette` 0.2 for gamepads; about 21.7k lines, 9.4k of them in `launcher.py`
Python deps	requests, pillow, vdf, psutil, icoextract
Build	Meson

Field	Value
Packaging	Flathub (x86_64 only, about 18.9k downloads per month), AUR, Fedora COPR, Ubuntu PPA, .deb, .rpm, AppImage
Translations	Weblate, 29 languages
Activity	One maintainer; one open issue at time of research

UI and navigation

Faugus puts one game model behind four presentations and keeps every control in a bottom bar; its Carousel mode is the closest thing to an Xbox-style hero view among the desktop references.

Main window

- `Gtk.ApplicationWindow` with no header bar by default (an option adds a plain `Gtk.HeaderBar`).
- Bottom bar: Add, Settings, Search entry, Kill-all, Play/Stop. Sort (Alphabetical, Playtime, Last played, Custom) and Category (All, Uncategorized, user categories) sit bottom right. A 50–100% zoom slider appears in Covers and Carousel; an option auto-hides the bars.
- Search is a case-insensitive substring match on title only; Custom sort is drag and drop.

View modes

Mode	Layout	Notes
List	Icon + title rows	Fixed, non-resizable narrow window (610 px high)
Grid	Icon tiles with labels	`Gtk.FlowBox`
Covers	2:3 poster grid with shadows	Selected cover scales to 1.05 with an accent glow
Carousel	Fanned row of covers, centre enlarged, full-bleed hero banner behind	Overview line: title, categories, playtime, last played

Non-list modes open at 1280×720 and can start maximized, fullscreen or at the remembered size. Interface options cover grid position, scroll orientation, rows or columns, overview panel colour, banner, labels, background (solid or dominant cover colour), interface sounds and a theme engine (Adwaita, system, or a GTK theme).

Console mode (`--console`): forces Carousel, fullscreen and gamepad navigation through libmanette. A confirms, B cancels, Y opens the game menu, X kills, LB adds, RB opens settings, Menu opens power options; Esc in fullscreen opens a power menu (shutdown, reboot, close).

Context menu: Play or Stop, Play with logs, Edit, Delete, Duplicate, Hide, Category, Open game location, Open prefix location, Run file in prefix, Recent files, Show logs; plus info lines for playtime, last played and "Playing for".

Running state: the Play button turns into Stop for the selected running game and the overview shows "Playing for"; the window polls `running-games.json` every second. There is no per-tile running badge.

Add/Edit dialog

- Banner on top, General and Settings tabs, cover preview on the right.
- General: type (Windows, Linux, Steam game, or a store launcher: Amazon, Battle.net, EA, Epic, GOG Galaxy, Rockstar, Ubisoft, Wargaming), title with SteamGridDB suggestions, path (pick executable or run installer), Steam Linux Runtime for Linux games, prefix (auto `<default>/<slug>`), runner, and shortcut checkboxes (desktop, app menu, Steam).

- Settings pages: Tools (game arguments, pre/post-launch scripts, MangoHud, GameMode, Discord RPC, No Sleep, SDL, Winetricks, Winecfg), Additional application, Lossless Scaling frame generation, and an ordered list of launch-argument tokens with presets.
- Art: right-click cover, banner or icon for Refresh, Load from file, Load from URL, Remove; left-click opens a SteamGridDB picker.

Settings dialog: General (default prefix location, default Proton, Proton Manager, autostart, tray, auto-close on launch, gamepad navigation, Wayland driver and WOW64 toggles, GPU selector, global environment variables, logs, language, backup and restore) and Interface tabs.

Other UI: a StatusNotifierItem tray process with the five most recent games; a Proton Manager with tabs for Proton-CachyOS, GE-Proton, DW-Proton and Proton-Wineland, each release downloadable or removable. Shortcuts: Enter plays, Delete deletes, Ctrl+H toggles hidden, Alt+Enter toggles fullscreen.

Features

Faugus builds each launch as an environment-plus-command string around its own umu-run, with per-game prefixes and auto-updated Proton builds.

Launch pipeline

1. `build_launch_command` assembles `FAUGUSID=<id> LOG_DIR=<id> GAMEID=umu-faugus<crc32> WINEPREFIX=… PROTONPATH=…`, then Lossless Scaling variables, the user's launch arguments, `gamemoderun` and `mangohud` if enabled, then `<umu-run> <exe> <game args>`.
2. `python -m faugus.runner` splits the string with `shlex` and executes it.
3. The window class becomes `steam_app_faugus<crc>`, so each game groups separately in the taskbar (new in 2.5.7).

Runners and prefixes

- Faugus downloads its own umu-run fork (`Faugus/umu-launcher`) to `~/.local/share/faugus-launcher/umu-run` before launches unless auto-update is off, plus EAC and BattlEye runtimes; it sets `UMU_USE_STEAM=1` and `UMU_CONTAINER_NSENTER=1`.

- Runners: Proton-CachyOS (default), GE-Proton, DW-Proton, Proton-Wineland "Latest" builds auto-updated into `compatibilitytools.d`; any other folder there; Flatpak Steam's folder (labelled); `Linux-Native` (optionally inside a Steam Linux Runtime); `Steam` (`steam -applaunch`).
- One prefix per game at `~/Faugus/<gameid>` by default, shareable; bare .exe files run in `~/Faugus/default`.
- Store launchers: Faugus downloads and installs the official Battle.net, EA, Epic, GOG and other installers into a prefix.

Toggles

- MangoHud, GameMode, SDL (`PROTON_PREFER_SDL`), No Sleep (portal Inhibit, falling back to logind), Discord RPC.
- Wayland driver (`PROTON_ENABLE_WAYLAND`), WOW64 (`PROTON_USE_WOW64`), GPU selection (`DRI_PRIME`, `MESA_VK_DEVICE_SELECT`, NVIDIA offload).
- No dedicated DXVK, VKD3D or Gamescope toggles: those are set as free-form environment variables; Gamescope is stated not to work in the Flatpak.

Integration features

- Desktop and app-menu shortcuts with `Exec=faugus-launcher --game <id>`.
- Steam non-Steam shortcuts written into `shortcuts.vdf`, with cover copied to `grid/<appid>p.png` and banner to `grid/<appid>_hero.png`.
- MIME handling for .exe, .msi, .lnk, .bat, .reg; drag and drop onto the window starts Add Game; watches for new `.lnk` files after an installer runs.
- Title and icon extracted from the .exe (ProductName and `icoextract`).
- Steam import from `appmanifest_*.acf`, Steam playtime from `localconfig.vdf`, Steam user selector.

Bookkeeping

- Playtime per game in seconds, saved every 30 s and on exit; `last_played` as an ISO timestamp. With auto-close on launch, running games are not tracked.
- Logs per game under `logs/<gameid>/` via "Play with logs".
- Backup of games, prefixes, shortcuts and Protons, with a daily or weekly daemon and 20 rolling `games-data-<ts>.json` copies.

Data, storage and integration points

An importer can read `games.json` and the art PNGs, launch with `--game <id>`, and read running state from `running-games.json`. Paths come from `path_manager.py`; in the Flatpak they resolve under `~/.var/app/io.github.Faugus.faugus-launcher/{config,data,.local/state}`.

Path	Contents
`~/.local/share/faugus-launcher/games.json`	JSON array of games (legacy `~/.config/faugus-launcher/games.json` is auto-migrated, so do not rely on it)
`~/.local/share/faugus-launcher/{covers,banners,icons}/<gameid>.png`	Covers 2:3 (SteamGridDB grids, 600×900); banners 1920:620 (SteamGridDB heroes); icons
`~/.local/share/faugus-launcher/categories.json`, `custom-order.json`, `presets.json`	Categories, manual sort order, launch-argument presets
`~/.config/faugus-launcher/config.json`	Flat settings object, booleans mostly stored as "True"/"False" strings
`~/.config/faugus-launcher/envar.json`	List of `KEY=VAL` global environment variables
`~/.local/state/faugus-launcher/running-games.json`	`{gameid: pid}` for running games
`~/.local/state/faugus-launcher/stop-<gameid>`	Request file that stops a game (internal, unstable)
`~/Faugus/<gameid>`	Wine prefixes
`~/.local/share/Steam/compatibilitytools.d/`	Proton builds

Game fields in `games.json`: `gameid` (title slug, with `-2`, `-3` for duplicates), `title`, `path`, `prefix`, `runner` (name, path, "Default", "Steam" or "Linux-Native"), `launch_arguments`, `game_arguments`, tool flags (`mangohud`, `gamemode`, `sdl_enabled`, `discord_rpc`, `disable_umu`, stored as `true` or `""`), additional-app fields, `cover` and

`icon` (absolute paths), `lossless_*`, `playtime` (int seconds), `hidden` (bool), `no_sleep`, `category` (string, list or `false`), `steamgriddb_id`, `pre_launch`, `post_launch`, `steam_user`, `runtime`, `last_played` (ISO). Writes are atomic (temp file then rename).

CLI (`faugus-launcher` wrapper)

- `--game <id>`: launch without the main window, with playtime tracking and splash.
- `--shortcut <file>`, `--run "<cmd>"`, `--daemon` (backup), `--hide`, `--console`; a bare file argument runs it in the default prefix.
- From another sandbox: `flatpak run --command=/app/bin/faugus-launcher io.github.Faugus.faugus-launcher --game <id>`.

There is no D-Bus API for games; the tray's D-Bus name only exposes Quit, Restart, Shutdown, RefreshMenu and Open.

Flatpak packaging and permissions

Faugus runs umu and Proton inside its own sandbox, which needs a wide permission set: `--device=all`, home access, multiarch and the i386 compat extensions. Manifest: flathub/io.github.Faugus.faugus-launcher, built from the 2.5.7 tag.

Item	Value
Runtime	`org.gnome.Platform` 51
Extensions	`org.freedesktop.Platform.Compat.i386` (26.08), GL32, VAAPI.Intel.i386; x86_64 only
Sockets and shares	network, ipc, wayland, x11, pulseaudio, `xdg-run/pipewire-0`
Allow	`devel` (needed by Ubisoft and other launchers), `bluetooth`, `multiarch`, `per-app-dev-shm` (pressure-vessel)
Devices	`--device=all`
Bus names	talk `org.kde.StatusNotifierWatcher`; system talk `org.freedesktop.UDisks2`; own `com.steampowered.*` (lets games share a prefix)

Item	Value
Filesystem	`home`, `/mnt`, `/run/media`, `xdg-config/MangoHud:ro`, `xdg-run/app/com.discordapp.Discord`
Environment	`PATH` extended with MangoHud, gamescope and OBS Vulkan capture extension dirs; `APPIMAGE_EXTRACT_AND_RUN=1`

Known Flatpak limits

- Gamescope does not work (stated in metainfo, README and on Flathub).
- MangoHud needs the `org.freedesktop.Platform.VulkanLayer.MangoHud` extension.
- Steam integration needs manual overrides on both sides: Faugus needs `~/.var/app/com.valvesoftware.Steam/` and `org.freedesktop.Flatpak`; Steam needs `org.freedesktop.Flatpak` and access to Faugus' config. Steam games and shortcuts then go through `flatpak-spawn --host`.
- The PID liveness check is skipped inside the Flatpak.

Adopt, adapt, avoid

Take Faugus' feature set for standalone Windows games and its hero-banner presentation, and rebuild it with real libadwaita widgets, typed settings and list models.

Adopt

- One game model with several presentations; the Carousel's hero banner plus overview line (categories, playtime, last played) is very close to the Xbox library look.
- SteamGridDB with a user API key, title autocomplete, and separate pickers for cover, banner and icon, each with Refresh, File, URL and Remove.
- Title and icon from exe metadata, drag and drop to add, "Run an installer" then detect the executable.
- Steam shortcut export including grid and hero art.
- A unique window class per game, playtime saved every 30 s, atomic JSON writes, rolling library backups.
- A hidden MIME handler entry for "Add with our app" from the file manager.

Adapt

- Rebuild dialogs as `AdwPreferencesDialog` and `AdwDialog` with `AdwEntryRow`, `AdwComboRow`, `AdwSwitchRow`.
- Replace the bottom bar with an `AdwHeaderBar` and search; switch view modes with a view switcher or toggle group.
- Use `GtkGridView` over a list model instead of `FlowBox`.
- Model categories as a proper list and booleans as JSON booleans.
- Import Faugus' `games.json`; reuse `compatibilitytools.d` for Proton builds and launch through the system umu-launcher.
- Turn free-text launch arguments into typed toggles (DXVK, VKD3D, Gamescope, MangoHud, GameMode), with a free-text escape hatch.

Avoid

- A monolithic main file and shell-string commands re-split with `shlex`; build argv lists.
- Silently downloading umu, anti-cheat runtimes and Proton at launch time.
- Fixed-size windows and a theme engine that fights libadwaita.
- Donation prompts tied to launching.

Open questions

- Does `HOST_XDG_DATA_HOME` reliably point Proton downloads at the host `compatibilitytools.d` from the Flatpak?
- Does HDR support go beyond the Lossless Scaling HDR mode?
- Can gamescope be made to work from a Flatpak launcher at all?

Sources

- Faugus/faugus-launcher (README), Releases, Release 2.5.0, Issues, #739
- Source files at commit 13b6df6: path_manager.py, config_manager.py, launcher.py, runner.py, core.py, components.py, proton_manager.py, tray_only.py, shortcut.py, steam_setup.py, metainfo.xml
- Flathub manifest repo
- Flathub app page
- GamingOnLinux: Faugus Launcher 2.0 rolls out with a new UI
- UbuntuHandbook: Faugus adds AppImage, GTK4 port and GOG Galaxy support